

 README

Project 5 - Challenge 2

In this assignment you will be fuzzing a real world target to find, root-cause and fix vulnerabilities. The target will be an old version of objdump from 2003. The setup is pretty much entirely hand-held in this part, so as long as you follow the steps carefully, you should be able to get the fuzzer running without much trouble.

Your goal will be to find 2 separate bugs, and implement a fix for both. Once you have a couple of initial crashes, chose one, fix the vulnerability, and recompile the binary for the fuzzer. Now clear the previously found crashes, and start fuzzing again to find a second bug to repeat the previous process on.

Submit a writeup that contains relevant screenshots and code examples to showcase that you successfully achieved the goals listed below.

```
20% Setup environment and run afl to find a crashing input
40% Triage crash to find and fix the first bug
40% Recompile with the previous fix, fuzz to find a second bug, and triage
    fix this bug
```

Install AFL

```
git clone https://github.com/AFLplusplus/AFLplusplus
cd AFLplusplus
make source-only
```

Install Objdump

```
mkdir -p targets/objdump_fuzzing && cd targets
wget https://ftp.gnu.org/gnu/binutils/binutils-2.14.tar.gz
tar xf binutils-2.14 && cd binutils-2.14
```

Compiling target

When compiling the target, there are 3 main options that we care about. The by default

generated binary, the afl-instrumented binary, and a binary compiled with debug information. The instrumented binary is used for the actual fuzzing process. Once the fuzzer finds a bug, the debug binary is used to triage the crash in a debugger. Finally once we finished triaging the crash, and fixed the bug in the source code of the binary, we can recompile it using the default config to make sure that our bugfix was actually successful. The commands used to compile the target are listed below. The main idea is to set environment variables that the makefile will use as its options.

2 things to note about compiling this target:

1. Between builds, always run `make clean` to remove prior binaries, otherwise you will run into issues.
2. When attempting to compile `binutils`, the compiler will eventually error out on something along the lines of `all-gas, Error 2`. You can just ignore this error, our target, the `objdump` binary has been successfully compiled and is located at:
`./binutils-2.14/binutils/objdump`.

Default build:

```
./configure --prefix=~/.cs390r/targets/binutils-2.14/build
make
```

With AFL instrumentation:

```
CC=~/.cs390r/AFLplusplus/afl-clang-fast ./configure \
--prefix=~/.cs390r/targets/binutils-2.14/build
make
```

With debug information:

```
CFLAGS="-g -O0" ./configure --prefix=~/.cs390r/targets/binutils-2.14/bui
make
```

Fuzzing Target

Now that we have both the target and afl setup we can start fuzzing. We will start by copying the target into our working directory. Next we setup a small corpus consisting of 4 elf binaries, and that's already all we need to do. We can now run afl against the targets using our specified corpus. The given options are explained below.

```
cd ~/.cs390r/targets/objdump_fuzzing
```

```
cp ../binutils-2.14/binutils/objdump ./objdump afl

# Setup inputs
mkdir in
cp /usr/bin/grep in
cp /usr/bin/ls in
cp /usr/bin/diff in
cp /usr/bin/dmesg in

AFL_SKIP_CPUFREQ=1 AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1 \
~/cs390r/AFLplusplus/afl-fuzz -i in -o out -- ./objdump afl -x @@
```

Command Explanation:

- `AFL_SKIP_CPUFREQ=1`
AFL likes to check that your kernel settings are setup in a way that is efficient for fuzzing. If they are not, it refuses to run by default and asks you to change your config. If you are running on your own computer, feel free to leave out this flag and instead modify the listed configs, if you are on the edlab however, you will require this flag since you don't have root privs.
- `AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1`
Same as above
- `-i in`
Directory containing corpus files
- `-o out`
Directory to which AFL will save output files
- `@@`
Placeholder in targets commandline that afl will substitute with file names
- `-x`
Objdump supports multiple different flags that generate different outputs. In this case we are fuzzing the '-x' flag.

Triaging/Fixing Crash

There are many much more sophisticated ways of doing this, but in our case we will just be doing it the manual way, with a text editor and gdb. Make sure you have a debug build of the target, and load the crashing input. Make sure to use the -x flag here as well.

Using a combination of the source-code and your debugger you should now be able to figure out what causes the bug to fix the vulnerability. (GDB should display the source

code file/line). Once you have an idea of how to fix the bug, implement the fix in source code, recompile the binary and see if the crash has been fixed (or you at least get a different crash this time).

The fixes don't need to attempt to continue execution if corruption is detected. Once you figure out how the bug occurs, add in a check that prints out an error message if a corruption occurs, and then proceeds to exit the program.

Rules for your fix: Your fix should make sense and pertain to the bug at hand, don't just ignore the bug and implement some other "fix" that causes the program to no longer segfault, but also not work properly. This means that after you apply your fix, it should no longer segfault on the crash, but it should still work perfectly fine on non-corrupted inputs.

If you are using vim, consider using something like ctags to enable jumping between source code program symbols. Additionally here are some gdb commands that you may find useful for this project:

```
backtrace    - print out function call stack
f <num>      - Change to a different function frame using a number from the
ptype <var>  - print out struct information
```